UNIT 1 · TOPIC 1.1

Understanding Social Engineering

Assignment

Instructions: 14 practice questions + 3 coding assignment(s), 28 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

1.1.a · Psychological tactics (intimidation, urgency, authority, scarcity)

1. An email says 'Your account closes in 1 hour unless you verify now.' Which tactic is this PRIMARILY using? (1 pt)

- ○ A. Scarcity
- ○ B. Reciprocity
- ○ C. Authority
- ○ D. Urgency

2. A message reads: 'This is the IRS. Pay the penalty within 24 hours or face arrest.' Select ALL tactics present. (3 pts)

- □ A. Authority
- □ B. Urgency
- □ C. Intimidation
- □ D. Scarcity

(Select ALL that apply.)

3. Why is recognizing the underlying TACTIC more useful than memorizing specific scam templates? (1 pt)

- ○ A. Scam templates are copyrighted, so security training cannot show real examples
- ○ B. Tactics stay constant even when attackers change the brand, wording, or channel
- ○ C. Spam filters already catch every known template, so only tactics are left to learn
- ○ D. Each tactic appears in only one kind of scam, so naming it identifies the scam

4. Name the single best defensive response when you notice manufactured emotional pressure in a message. (1 pt)

1.1.b · Elicitation; malicious links and files

5. An attacker tells a receptionist 'I heard your VPN uses port 1433, or was it 443?' hoping the receptionist corrects them. Which elicitation gambit is this? (1 pt)

- ○ A. Deliberate incorrect statement
- ○ B. Flattery / appeal to expertise
- ○ C. Mutual disclosure

○ D. Third-party reference

6. Which of the following are malicious indicators in the URL http://paypa1.com/verify.exe ? Select ALL that apply. (2 pts)

□ A. lookalike-domain (paypa1.com imitates paypal.com)

□ B. mismatched-link (the link text differs from where it really goes)

□ C. dangerous-extension (.exe)

□ D. ip-url (the host is a numeric address)

(Select ALL that apply.)

7. A filename shows as 'invoice.pdf' in Windows Explorer. When you look at the true name it is 'invoice.pdf.exe'. What technique does this use? (1 pt)

○ A. Typosquatting

○ B. Double extension

○ C. Homograph attack

○ D. URL shortening

8. Why does HTTPS in a URL NOT prove that a website is safe? (1 pt)

9. Which action BEST defends against elicitation when someone you do not recognize asks detailed questions about your network? (1 pt)

○ A. Answer only general questions and refuse anything specific

○ B. Ask them a security question only a real employee would know

○ C. Verify their identity through official channels before sharing any operational details

○ D. Share only details that are already on the company website

1.1.c · Email, text, and social media attacks

10. An attacker registers 'paypa1.com' (digit 1 replacing the letter l) and sends password-reset emails from that domain. Which technique does this PRIMARILY represent? (1 pt)

○ A. Domain lookalike / homograph attack

○ B. Spear phishing

○ C. Smishing

○ D. Watering-hole attack

11. A text message says: 'UPS: Your package is held. Reschedule at http://bit.ly/upsdel.' What makes SMS (smishing) especially effective for this type of attack? (1 pt)

○ A. SMS messages cannot be filtered by spam engines

○ B. Limited screen real estate hides full URLs; people trust texts on personal devices

○ C. Phones display URLs in full without truncation

○ D. SMS is encrypted end-to-end so attackers can bypass firewalls

12. A social media DM says: 'Your account expires tonight. Verify at http://192.168.5.1/login to avoid losing access.' Select ALL phishing indicators present. (3 pts)

□ A. Shortened link / bare IP address URL

□ B. Urgency (account expiring tonight)

□ C. Credential request (verify to avoid losing access)

□ D. Legitimate organizational sender

(Select ALL that apply.)

13. What distinguishes spear phishing from generic phishing? (1 pt)

○ A. Spear phishing is sent by text message, while phishing is sent by email

○ B. Spear phishing always carries malware, while phishing only steals passwords

○ C. Spear phishing targets executives, while phishing targets ordinary employees

○ D. Spear phishing uses pre-researched details about a specific target to increase believability

14. Name the one defensive action that works across email, SMS, and social media phishing when you suspect a message is fraudulent. (1 pt)

Coding Assignments (Python 3)

1.1.a · Psychological tactics (intimidation, urgency, authority, scarcity)

3 pts · auto-graded on the site

Coding problem 1.1.a: Detect psychological pressure tactics

Skill: Analyze Risk · **Maps to:** 1.1.A, 1.1.B · **Language:** Python 3 (pytest)

Problem

Build the detector a phishing-awareness tool would use. Given the raw text of a message, return which of the four psychological pressure tactics it employs:

- `"urgency"`: manufactured time pressure
- `"authority"`: borrowed legitimacy (boss, IT, government, bank)
- `"scarcity"`: limited availability / fear of missing out
- `"intimidation"`: threats / fear

A message may use zero, one, or several tactics.

Function contract

Implement `solve(text: str) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the detected tactic labels as a **sorted, de-duplicated** list. Matching is case-insensitive. Detection is by the indicator phrase sets defined in `interface.md`.

Input

A single string, the message body (and/or subject) to analyze.

Output

A sorted list of unique tactic labels from the set above (empty list if none).

Examples

```
"Your account will be locked in 24 hours. Act now."        -> ["intimidation", "urgency"]
"This is the IT department. Limited spots — last chance."  -> ["authority", "scarcity"]
"Pay the penalty or we will take legal action immediately" -> ["intimidation", "urgency"]
"Lunch is in the kitchen, help yourself."                  -> []
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.1.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List


def solve(text: str) -> List[str]:
    """Return the sorted, de-duplicated psychological tactics present in `text`.

    Tactics: "authority", "intimidation", "scarcity", "urgency".
    See prompt.md and interface.md for the indicator phrase sets.
    """
    # TODO: scan `text` (case-insensitive) for each tactic's indicator phrases
    #       and return a sorted, de-duplicated list of the tactics found.
    raise NotImplementedError("implement solve() — see prompt.md")
```

1.1.b · Elicitation; malicious links and files

3 pts · auto-graded on the site

Coding problem 1.1.b: Malicious link and file indicator detector

Skill: Analyze Risk · **Maps to:** 1.1.A, 1.1.C · **Language:** Python 3 (pytest)

Problem

Build the analyzer a phishing-awareness tool would use to flag suspicious URLs and filenames. Given a single string that is either a URL or a filename, return which malicious indicators it exhibits.

Four indicator categories are defined:

- `"lookalike-domain"`: the URL hostname resembles a known brand but is not the real domain (typosquatting rules defined in `interface.md`)
- `"ip-url"`: the URL uses a raw IPv4 address as its host instead of a domain name
- `"dangerous-extension"`: the filename (or URL path) ends in an executable or script extension
- `"mismatched-link"`: the URL contains a brand keyword in a path or subdomain while the actual registrable domain belongs to a different host (defined precisely in `interface.md`)

Function contract

Implement `solve(url_or_filename: str) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Return the detected indicator labels as a **sorted, de-duplicated** list.

Input

A single string, either a URL (starting with `http://` or `https://`) or a bare filename.

Output

A sorted list of zero or more indicator labels drawn from {`"dangerous-extension"`, `"ip-url"`, `"lookalike-domain"`, `"mismatched-link"`}.

Examples

```
"http://paypa1.com/verify"                  -> ["lookalike-domain"]
"http://192.168.4.99/reset"                 -> ["ip-url"]
"invoice.pdf.exe"                           -> ["dangerous-extension"]
"http://bankofamerica.com.secure-login.xyz" -> ["mismatched-link"]
"http://shop.example/track/44821"           -> []
"http://dropbx.com/s/Q3.scr"               -> ["dangerous-extension", "lookalike-domain"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.1.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List


def solve(url_or_filename: str) -> List[str]:
    """Return sorted, de-duplicated malicious indicators present in url_or_filename.

    Indicators: "dangerous-extension", "ip-url", "lookalike-domain", "mismatched-link".
    See prompt.md and interface.md for exact detection rules.

    Args:
        url_or_filename: A URL (starting with http:// or https://) or a bare filename.
    Returns:
        Sorted list of indicator labels; empty list if none detected.
    """
    # TODO: implement per prompt.md and interface.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

1.1.c · Email, text, and social media attacks

3 pts · auto-graded on the site

Coding problem 1.1.c: Phishing indicator extractor

Skill: Analyze Risk, Detect Attacks · **Maps to:** 1.1.A, 1.1.C · **Language:** Python 3 (pytest)

Problem

Build the indicator extractor a security-awareness tool would use to screen incoming messages across channels. Given a message dict with channel, sender, text, and link fields, return the sorted list of phishing indicator labels that the message triggers.

A message may trigger zero, one, or several indicators. The result is the sorted, de-duplicated list of triggered label strings.

Function contract

Implement `solve(message: dict) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A dict with the following keys (all strings):

- `"channel"`: one of `"email"`, `"sms"`, `"dm"` (social media direct message)
- `"sender"`: the sender address, phone number, or handle
- `"text"`: the message body text
- `"link"`: a URL or empty string

Output

A **sorted, de-duplicated** `list[str]` drawn from the five indicator labels:

- `"lookalike-domain"`: sender contains a mangled brand domain
- `"urgent-money-request"`: text requests an urgent financial transaction
- `"credential-request"`: text requests credentials or account verification
- `"mismatched-sender"`: text claims a brand the sender address does not belong to
- `"shortened-link"`: link uses a URL shortener or bare IP address

See `interface.md` for the precise indicator rule sets.

Examples

```
{
  "channel": "email",
  "sender": "security@paypa1.com",
  "text": "Verify your PayPal account now.",
  "link": ""
}
-> ["credential-request", "lookalike-domain", "mismatched-sender"]

{
  "channel": "sms",
  "sender": "+1-555-0192",
  "text": "Reminder: dentist at 2pm tomorrow. Reply YES.",
  "link": ""
}
-> []

{
  "channel": "email",
  "sender": "ceo@corp-executive.net",
  "text": "Wire $50k today, do not loop in finance, keep this confidential.",
```

```
  "link": ""
}
-> ["urgent-money-request"]

{
  "channel": "dm",
  "sender": "@IT_support99",
  "text": "Your account expires tonight, verify here.",
  "link": "http://192.168.1.1/login"
}
-> ["credential-request", "shortened-link"]

{
  "channel": "sms",
  "sender": "NETFLIX",
  "text": "Your Netflix payment failed. Update your payment at netflix.com/billing",
  "link": "https://bit.ly/3xAmPle"
}
-> ["credential-request", "shortened-link"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.1.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List


def solve(message: dict) -> List[str]:
    """Return sorted, de-duplicated phishing indicator labels for `message`.

    Args:
        message: dict with keys channel, sender, text, link (all strings).
    Returns:
        Sorted list of triggered indicator label strings (may be empty).
        Labels: "lookalike-domain", "urgent-money-request", "credential-request",
                "mismatched-sender", "shortened-link".
        See prompt.md and interface.md for the precise detection rules.
    """
    # TODO: check each indicator rule against message["sender"], message["text"],
    #       and message["link"] (all case-insensitive), collect triggered labels,
    #       and return them sorted and de-duplicated.
    raise NotImplementedError("implement solve() — see prompt.md")
```